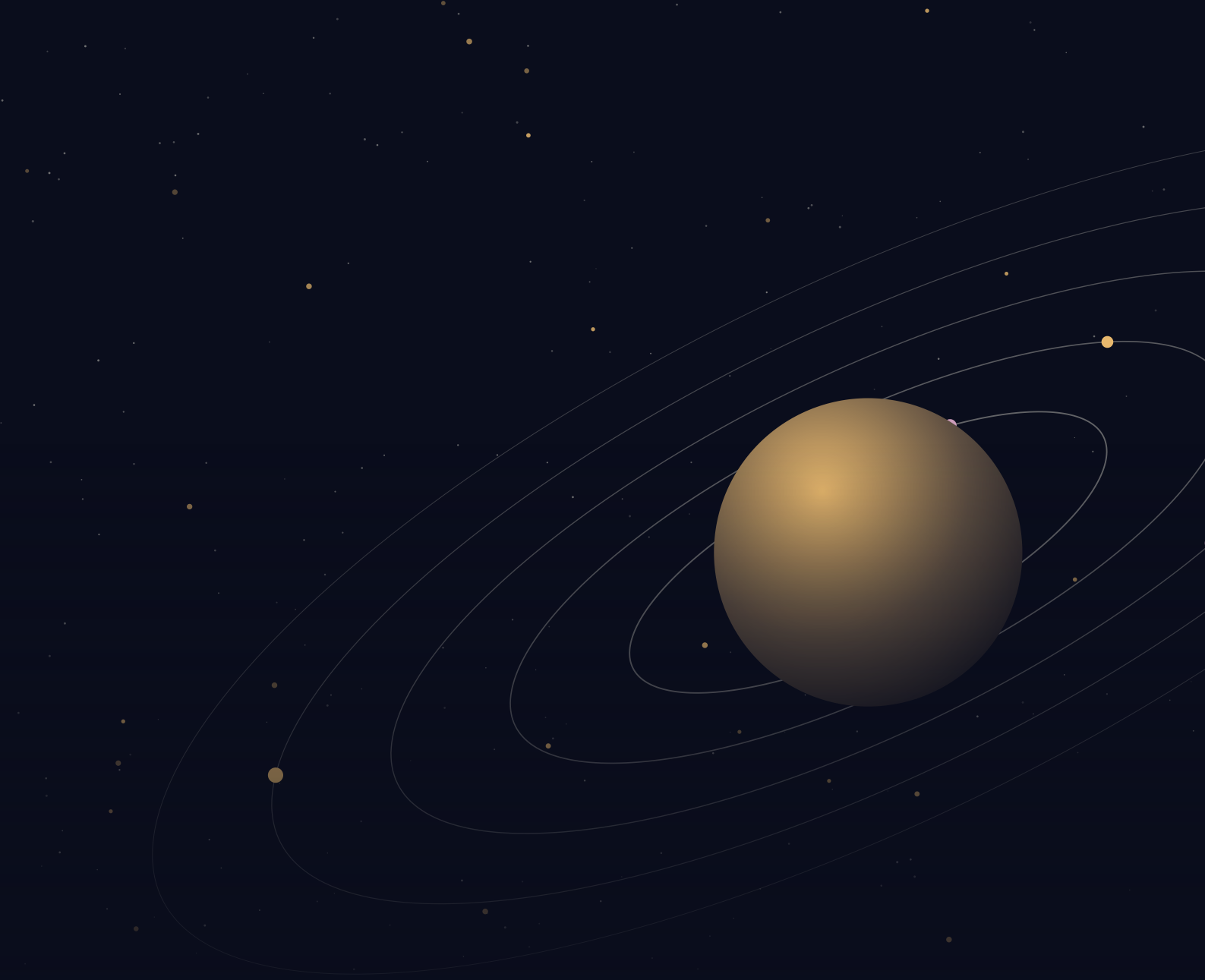


TUTORIAL 7 · 中文深色讲解版

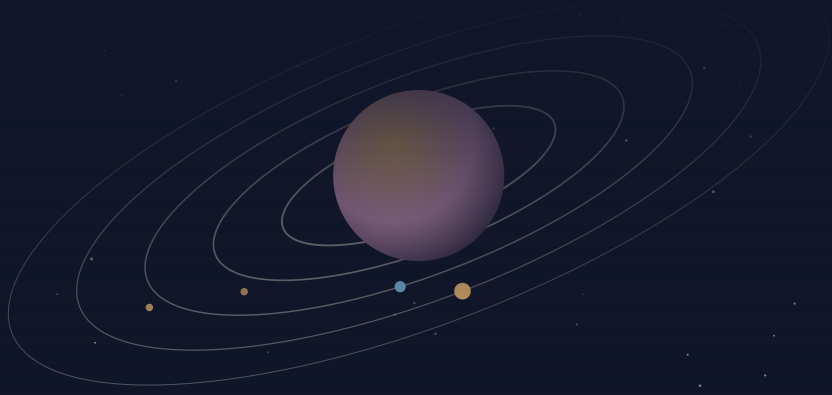
数值方法四讲

插值 · 求根 · 数值微分 · 数值积分

2026 年 11 月

目录

01	导读：四个主题，一条主线	3
02	插值：把离散点连成曲线	5
03	求根：解 $f(x)=0$	12
04	数值微分	19
05	数值积分	27
06	练习与完整解答	34
07	速查卡	38



01

导读：四个主题，一条主线

这一讲把四件看似无关的事放在一起：**插值、求根、数值微分、数值积分**。它们其实共用同一条主线——计算机手里只有**有限个离散数字**，而数学问题问的却是**连续对象**：曲线上任意一点的值、方程的精确零点、某一点的瞬时斜率、曲线下的精确面积。数值方法就是用有限信息去逼近无限精度，并且**对误差心中有数**。

要点

贯穿全讲的三个问题，做每一道题都要问一遍：① **我在用什么替代真实对象？**（直线？抛物线？切线？）② **这个替代带来的误差是多少阶？**（$\mathcal{O}(h)$ 还是 $\mathcal{O}(h^2)$、$\mathcal{O}(h^4)$？）③ **什么时候它会彻底失效？**（噪声、$f' \to 0$、根不存在、步长过小导致舍入误差爆炸）

四个主题的地图

表 1 四个主题的核心对照

主题	要回答的问题	替代物	主力方法	典型精度
插值	已知点之间的值是多少？	分段直线 / 分段三次 / 全局多项式	`interp1d`、`CubicSpline`、牛顿差商	取决于阶数与光滑性
求根	$f(x)=0$ 的解在哪？	区间折半 / 切线	二分法、牛顿–拉夫森、`fsolve`	线性 / 二次收敛
数值微分	这一点的斜率是多少？	割线	前向、后向、中心差分	$\mathcal{O}(h)$ / $\mathcal{O}(h^2)$
数值积分	曲线下面积是多少？	矩形 / 梯形 / 抛物线	黎曼和、中点、梯形、辛普森	$\mathcal{O}(h)\sim\mathcal{O}(h^4)$

150×
辛普森比梯形准
同样 11 个网格点

21 → 4
二分法 vs 牛顿法
达到 10^{-6} 所需迭代次数

1% → 100%
噪声被微分放大
$\epsilon\omega = 0.01 \times 100 = 1$

本讲所有代码只依赖两个库，先把它们导进来：

```python
import numpy as np
import matplotlib.pyplot as plt
```

> **注**
>
> 后面还会陆续用到 `scipy.interpolate`（插值）、`scipy.optimize`（求根）、`scipy.integrate`（积分）。SciPy 的函数是**生产级实现**，但作业与考试要求你能手写公式——理解原理之后再调库，才知道库什么时候会骗你。

02

插值：把离散点连成曲线

插值（interpolation）要解决的是：已知 n 个数据点

$$(x_1, y_1),\ (x_2, y_2),\ \ldots,\ (x_n, y_n)$$

求一个估计函数 $\hat{f}(x)$，使它**穿过每一个数据点**：

$$\hat{f}(x_i) = y_i \qquad \text{对数据集中的每个点都成立}$$

并用它来估计**落在已知数据范围之内**的未知值。

插值和回归不是一回事

这是最容易混的概念，务必分清：

插值 Interpolation

曲线**必须穿过每一个点**。 不假设背后有什么参数模型，只要求"过点 + 某种光滑性"。 适用：数据可靠、噪声极低、代表一个连续的物理过程（例如高精度设备记录的车辆轨迹）。 风险：数据里若有噪声，插值会**忠实地把噪声也复制进曲线**。

回归 Regression

先假设一个参数模型（例如直线 $y = mx + b$），再**最小化残差总和**。 拟合出的线通常**不经过任何一个点**。 适用：数据有测量噪声，你关心的是趋势而不是单点。 风险：模型选错了，再怎么拟合都是错的。

> **注意**
>
> 插值只在数据范围**内部**可信。超出 $[x_1, x_n]$ 叫**外插（extrapolation）**，误差会迅速失控，尤其是高次多项式——这一点在本章末尾的龙格现象里会看得非常清楚。

线性插值

最简单的做法：假设任意中间点就落在**左右两个最近已知点连成的直线段**上。对于 $x_i \le x \le x_{i+1}$：

$$\hat{f}(x) = y_i + \frac{y_{i+1} - y_i}{x_{i+1} - x_i}(x - x_i)$$

这就是两点式直线，只不过逐段使用。

```python
from scipy.interpolate import interp1d

# 已知的离散数据点
x_data = np.array([0, 1, 2])
y_data = np.array([1, 3, 2])

# 构造一维线性插值器：注意它返回的是一个"函数"
f_linear = interp1d(x_data, y_data, kind='linear')

# 在目标点 x = 1.5 处求值
x_target = 1.5
y_target = float(f_linear(x_target))
print(f"Linear Interpolation at x = {x_target}: y_hat = {y_target:.4f}")
```

输出：

```text
Linear Interpolation at x = 1.5: y_hat = 2.5000
```

手算验证：$x = 1.5$ 落在 $[1, 2]$ 区间，$\hat{f}(1.5) = 3 + \frac{2-3}{2-1}(1.5 - 1) = 3 - 0.5 = 2.5$ ✓

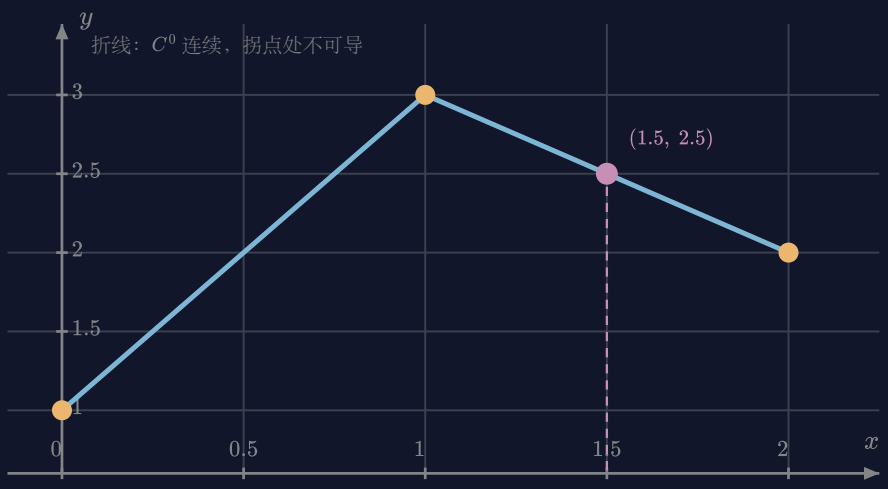


图 1 线性插值：三点连成折线，在 x=1.5 处读出 2.5

注意

线性插值的致命弱点就画在图里：曲线在每个数据点处有**尖角**，只有 C^0 连续。一阶导数在节点处跳变，二阶导数根本不存在。如果这条曲线代表机械臂的位置，那就意味着**速度突变、加速度无穷大**——物理上不可实现。

三次样条插值

为了修掉尖角，三次样条（cubic spline）在**每个区间上各用一个三次多项式**：

$$S_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i, \qquad x \in [x_i,\, x_{i+1}]$$

n 个点有 $n-1$ 个区间，共 $4(n-1)$ 个未知系数。靠下面三组条件把它们唯一钉死：

表 2 三次样条的约束计数（n 个数据点）

约束类型	含义	方程个数
过点条件	每段在两端都取到给定的 y 值	$2(n-1)$
一阶导连续	节点处斜率左右相等	$n-2$
二阶导连续	节点处曲率左右相等	$n-2$
边界条件	自然样条：两端 $S''=0$	2
合计		$4(n-1)$ ✓

"自然边界条件"（`bc_type='natural'`）就是让曲线在两端**自然放平**（$S''=0$），物理上相当于一根两端自由的弹性尺。SciPy 的 `CubicSpline` 还支持 `'clamped'`（指定端点斜率）、`'not-a-knot'`（默认）等边界条件。

```python
from scipy.interpolate import CubicSpline

x_data = np.array([0, 1, 2])
y_data = np.array([1, 3, 2])

# 自然三次样条：bc_type='natural' 强制两端 S''(x) = 0
f_spline = CubicSpline(x_data, y_data, bc_type='natural')

y_target_spline = float(f_spline(1.5))
print(f"Cubic Spline at x = 1.5: y_hat = {y_target_spline:.5f}")
```

输出：

```
Cubic Spline at x = 1.5: y_hat = 2.78125
```
text

为什么是 2.78125，而不是 2.5?

值得手算一遍，因为这解释了样条的"性格"。设二阶导在节点处的值为 M_0, M_1, M_2，自然边界给出 $M_0 = M_2 = 0$，内点方程（等距 $h = 1$）为

$$\frac{2h}{3} M_1 = \frac{y_2 - y_1}{h} - \frac{y_1 - y_0}{h} = (-1) - (2) = -3 \implies M_1 = -4.5$$

代回分段公式，得到两段显式三次式：

$$S(x) = \begin{cases} 1 + 2.75x - 0.75x^3, & x \in [0, 1] \\ 5.5 - 1.75x - 0.75(2 - x)^3, & x \in [1, 2] \end{cases}$$

于是 $S(1.5) = 5.5 - 2.625 - 0.75 \times 0.125 = 2.78125$ ✓

要点

样条值 $2.78125 > 2.5$ 不是算错了。为了让曲线在 $x = 1$ 的尖峰处**平滑地转弯**，它必须在峰顶附近"鼓"出去一点。这是光滑性的代价：**样条用轻微的过冲，换取连续的一阶与二阶导数。**

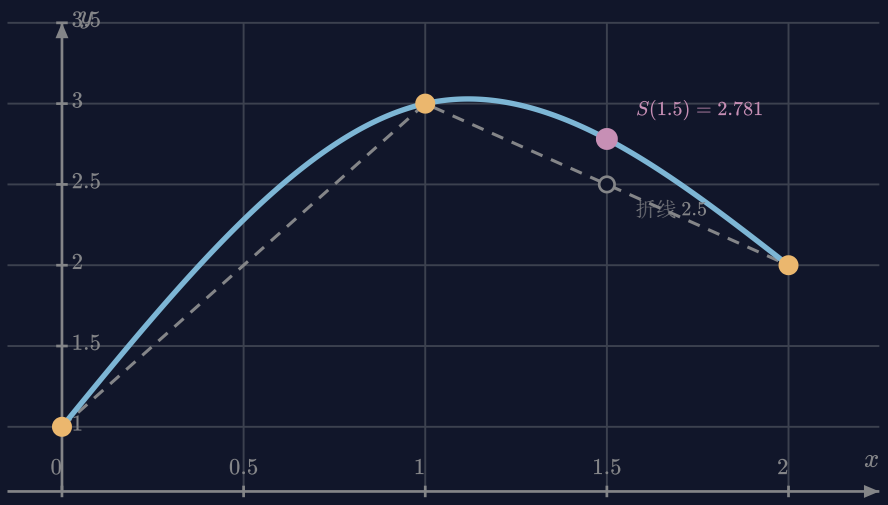


图 2 自然三次样条（实线）对比线性插值（虚线）：平滑过渡带来轻微过冲

拉格朗日多项式

另一条路：不用分段，直接找**一个**次数为 $n - 1$ 的全局多项式穿过全部 n 个点。拉格朗日把它写成基函数的线性组合：

$$L(x)=\sum_{i=1}^{n} y_i\,P_i(x),\qquad P_i(x)=\prod_{\substack{j=1\\ j\neq i}}^{n}\frac{x-x_j}{x_i-x_j}$$

每个基函数都有一个漂亮的性质：

$$P_i(x_j)=\begin{cases}1, & i=j\\ 0, & i\neq j\end{cases}$$

所以把 x_j 代进 L，只有第 j 项活下来，正好等于 y_j——过点条件自动满足，**不需要解任何方程组**。

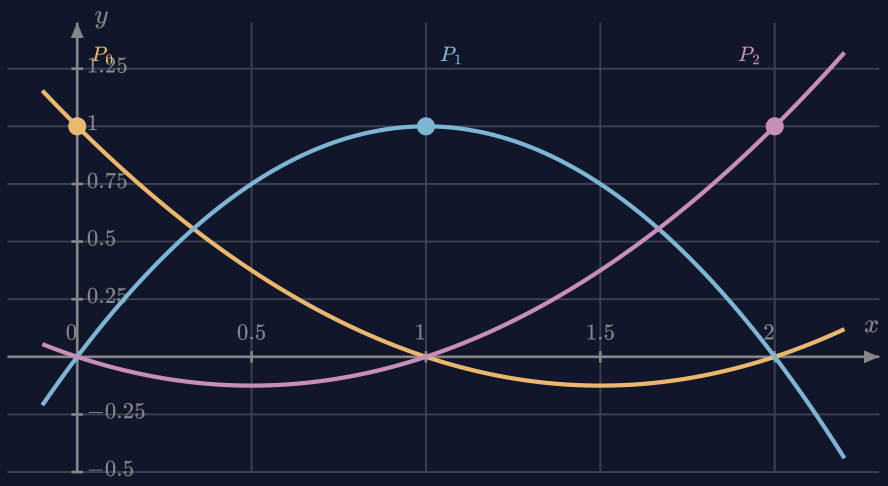


图 3 节点 x=0,1,2 上的三个拉格朗日基函数：各自在自己的节点取 1，在别人的节点取 0

牛顿多项式与差商

牛顿形式与拉格朗日**数学上完全等价**（过同样 n 个点的 $n-1$ 次多项式唯一），但计算上好用得多：**新增一个数据点时原有系数不必重算**，只要在末尾追加一项。

$$f(x)=a_0+a_1(x-x_0)+a_2(x-x_0)(x-x_1)+\cdots+a_n(x-x_0)\cdots(x-x_{n-1})$$

系数 a_i 由**差商表（divided differences）**给出：

$$f[x_0,x_1]=\frac{f(x_1)-f(x_0)}{x_1-x_0},\qquad f[x_0,x_1,x_2]=\frac{f[x_1,x_2]-f[x_0,x_1]}{x_2-x_0}$$

一般地 $a_k=f[x_0,\ldots,x_k]$，正好是差商表的**第一行**。

```python
def divided_diff(x, y):
    """
    计算坐标向量 x, y 的差商表。
    返回完整矩阵，其第一行即为牛顿多项式的系数 a_i。
    """
    n = len(y)
    coef = np.zeros([n, n])
    coef[:, 0] = y                          # 第一列就是 y 本身

    for j in range(1, n):                   # 逐阶往右推
        for i in range(n - j):
            coef[i][j] = (coef[i+1][j-1] - coef[i][j-1]) / (x[i+j] - x[i])
    return coef

def newton_poly(coef, x_data, x):
    """用 Horner 嵌套法求值，避免重复乘法。"""
    n = len(x_data) - 1
    p = coef[n]
    for k in range(1, n + 1):
        p = coef[n-k] + (x - x_data[n-k]) * p
    return p

x_pts = np.array([-5, -1, 0, 2])
y_pts = np.array([-2,  6, 1, 3])

table = divided_diff(x_pts, y_pts)
a_s = table[0, :]                           # 取第一行作为系数
```

算出的差商表（只有上三角有意义，第一行加粗的就是系数）：

表 3 数据 $(-5,-2),(-1,6),(0,1),(2,3)$ 的差商表

x_i	$f[x_i]$	一阶	二阶	三阶
-5	-2	$\mathbf{2}$	$\mathbf{-1.4}$	$\mathbf{0.485714}$
-1	6	-5	2	
0	1	1		
2	3			

于是

$$f(x) = -2 + 2(x+5) - 1.4(x+5)(x+1) + 0.485714\,(x+5)(x+1)x$$

展开后等于 $0.4857x^3 + 1.5143x^2 - 3.9714x + 1$，可以逐点验证它确实穿过四个数据点。

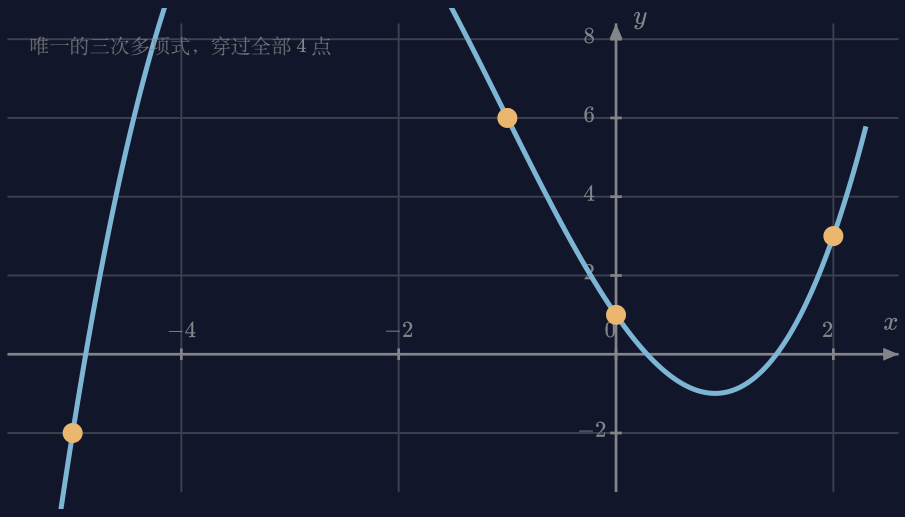


图 4 牛顿差商插值：四个点唯一确定一条三次曲线

补充：全局多项式的陷阱——龙格现象

既然一个全局多项式就能穿过所有点，工程上为什么还要费劲用分段样条？看这个经典反例：取 $f(x)=\dfrac{1}{1+25x^2}$，在 $[-1,1]$ 上用 **11 个等距节点**做 10 次插值。

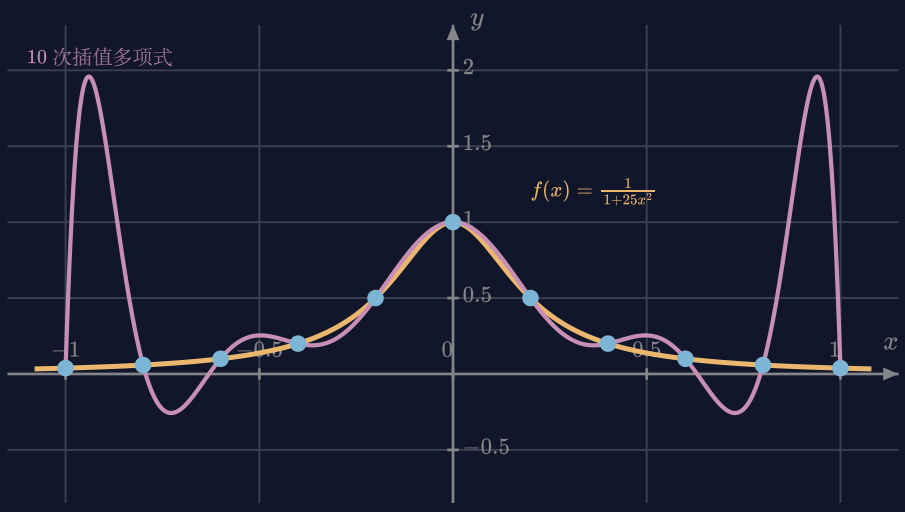


图 5 龙格现象：等距节点下，高次多项式在区间两端剧烈振荡，峰值已接近 2

注意

多项式次数越高 ≠ 越精确。 等距节点下，次数一高，插值多项式会在端点附近剧烈振荡，误差随节点增多反而发散。两条出路：① 改用**分段低次**方法（三次样条）——工程默认选择；② 改用**非等距节点**（切比雪夫节点，两端加密）。 一句话经验：**点多就分段，别硬提次数。**

03

求根：解 $f(x)=0$

函数 $f(x)$ 的根（零点）是满足

$$f(x_r)=0$$

的那个 x_r。简单代数方程（$x^2-9=0\Rightarrow x=\pm 3$）可以解析求解；但**超越方程**如 $f(x)=\cos x - x$ 没有闭式解，只能用迭代法。

而且计算机用的是**有限精度浮点数**，算法几乎永远不会刚好落在 $f(x)=0.000000\ldots$ 上。所以迭代法必须有**终止判据**：

判据一：函数残差	**判据二：步长**
$\lvert f(x_k)\rvert < \text{tol}$	$\lvert x_{k+1}-x_k\rvert < \text{tol}$
衡量"输出离零有多近"。陷阱：函数非常平坦时（$\lvert f'\rvert \ll 1$），残差很小但 x_k 可能离真根很远。	衡量"猜测值是否已经不再变化"。陷阱：收敛很慢时，相邻两步差很小也不代表已经到达根。

提示

稳妥做法是**两个判据一起用**，再加一个最大迭代次数上限，任一满足即停止。本讲的 `my_bisection` 就同时检查了残差和区间长度。

SciPy 的 fsolve

`fsolve` 接受一个函数和一个初始猜测；也可以指定容差与最大迭代次数，默认分别是 1.49012×10^{-8} 和 400。它还能解**方程组**（传函数数组和初值数组），或者对同一个函数传**多个初值**来找多个根。

```python
from scipy.optimize import fsolve

# 情形 1：成功——超越方程 cos(x) - x = 0
f_cos = lambda x: np.cos(x) - x
root_cos = fsolve(f_cos, x0=-2)
print(f"Converged root: r = {root_cos[0]:.8f}")
print(f"Residual f(r):  {f_cos(root_cos)[0]:.2e}")

# 情形 2：失败——f(x) = 1/x 根本没有根
f_inv = lambda x: 1.0 / x
r_inv, infodict, ier, mesg = fsolve(f_inv, x0=-2, full_output=True)
print(f"Result value:   {r_inv[0]:.4e}")
print(f"Residual f(r):  {f_inv(r_inv)[0]:.4e}")
print(f"Function calls: {infodict['nfev']}")
print(f"Exit message:   {mesg}")

# 情形 3：多个根——f(x) = (x^2-1)(x-100)，根为 -1, 1, 100
f_cubic = lambda x: x**3 - 100*x**2 - x + 100
guesses = [-5, 2, 80]
found_roots = [float(fsolve(f_cubic, g)[0]) for g in guesses]
print(f"Found roots: {found_roots}")
```

输出：

```text
--- Case 1: Success (cos(x) - x = 0) ---
Converged root: r = 0.73908513
Residual f(r):  0.00e+00

--- Case 2: Failure (1/x = 0) ---
Result value:   -3.5205e+83
Residual f(r):  -2.8405e-84
Function calls: 402
Exit message:   The number of calls to function has reached maxfev = 400.

--- Case 3: Multiple roots ((x^2 - 1)(x - 100) = 0) ---
Found roots: [-1.0000000000000022, 1.0, 100.00000000000001]
```

注意

情形 2 是整章最值得记住的一幕。 $1/x$ 没有零点，但 `fsolve` 返回了 -3.52×10^{83}，而且残差 -2.84×10^{-84} **小得离谱**。如果只看残差就宣布"收敛了"，就会把一个荒谬的数当成答案。

真正暴露问题的是另外两条信息：函数被调用了 **402** 次（撞上 `maxfev=400` 上限），以及退出信息明确写着"达到最大调用次数"。

结论：永远检查返回的 `ier` 标志和 `mesg`，而不是只看残差。 残差小只说明"函数值接近 0"；对于 $f\to0$ 却永不等于 0 的函数（$1/x$、e^{-x}），这毫无意义。

注

情形 3 说明：`fsolve` 一次只给**一个**根，给哪个取决于初值落在谁的"吸引域"里。想找全部根就得撒一把初值，或者先画图看看根大概在哪。另外注意返回值是 -1.0000000000000022 和 100.00000000000001——浮点运算下**绝不要用 `==` 比较根**。

二分法

二分法是一种**包围法（bracketing method）**，依据介值定理：若 $f(x)$ 在 $[a,b]$ 上连续且 $f(a)\cdot f(b)<0$（异号），则区间内**至少存在一个根** c 使 $f(c)=0$。

01 检查符号
确认 $f(a)$ 与 $f(b)$ 异号，即 $f(a)\cdot f(b)<0$；否则无法保证有根，直接报错。

02 取中点
$m=\dfrac{a+b}{2}$。

03 判停
若 $|f(m)|<\text{tol}$（或区间半长 $<\text{tol}$），接受 m 为根。

04 砍掉一半
若 $\operatorname{sign}f(a)=\operatorname{sign}f(m)$，根在 $[m,b]$，令 $a\leftarrow m$；否则根在 $[a,m]$，令 $b\leftarrow m$。

05 循环
重复直到区间长度或残差低于容差。

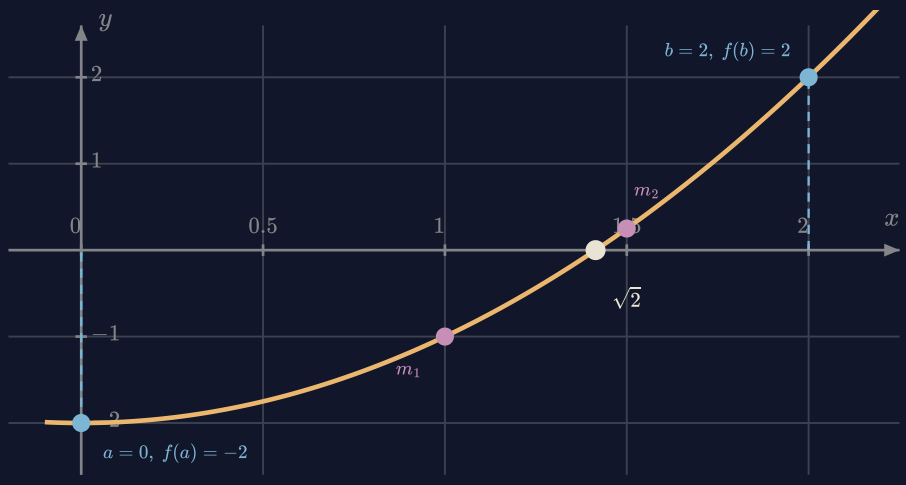


图 6 二分法求 √2：异号端点把根夹住，每一步把区间砍一半

```python
def my_bisection(f, a, b, tol):
    """在 [a, b] 内把 f 的根逼近到容差 tol 以内。"""
    # 两端同号则无法保证区间内有根，直接抛异常
    if np.sign(f(a)) == np.sign(f(b)):
        raise ValueError(f"The scalars a={a} and b={b} do not bound a root (same sign).")

    steps = 0
    while True:
        steps += 1
        m = (a + b) / 2.0                       # 当前区间中点

        if abs(f(m)) < tol or (b - a) / 2.0 < tol:
            return m, steps                     # 残差或区间半长达标即返回

        if np.sign(f(a)) == np.sign(f(m)):
            a = m                               # 中点与 a 同号 → 根在右半边
        else:
            b = m                               # 否则根在左半边

f_sqrt2 = lambda x: x**2 - 2

for tol_val in [0.1, 0.01, 1e-6]:
    root_val, iterations = my_bisection(f_sqrt2, 0, 2, tol_val)
    print(f"Tolerance = {tol_val:7g} → Root = {root_val:10.8f} in {iterations:2d} iterations")
```

输出：

```text
Tolerance =     0.1 → Root = 1.43750000 in  5 iterations (Exact: 1.41421356)
Tolerance =    0.01 → Root = 1.41406250 in  8 iterations (Exact: 1.41421356)
Tolerance =   1e-06 → Root = 1.41421413 in 21 iterations (Exact: 1.41421356)
```

读懂这张表：收敛速度是"线性"的

每迭代一次，区间长度正好减半：$|b_k - a_k| = \dfrac{b_0 - a_0}{2^k}$。要把误差从 10^{-1} 压到 10^{-6}，需要大约

$$k \approx \log_2 \frac{b_0 - a_0}{\text{tol}} = \log_2 \frac{2}{10^{-6}} \approx 21$$

次迭代——和输出里的 21 完全对上。这叫**线性收敛**：每一步让误差乘以固定因子 $\frac{1}{2}$，换算成十进制约等于**每 3.3 次迭代多一位有效数字**。

要点

二分法的交易条件非常清晰：**用速度换可靠性。** 只要初始区间异号且函数连续，它**一定**收敛——不会发散、不会振荡、不依赖导数。代价是慢，而且只能找**区间内**的根，也无法处理偶数重根（例如 $f(x) = x^2$ 在根处不变号，二分法根本起不了步）。

牛顿–拉夫森法

牛顿法用**局部切线**（一阶泰勒近似）去预测下一个猜测点：

$$f(x) \approx f(x_0) + f'(x_0)(x_1 - x_0) = 0$$

解出 x_1 就得到牛顿迭代式：

$$\boxed{x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}}$$

几何意义：在 x_k 处画切线，看它与 x 轴的交点在哪，那里就是下一个 x_{k+1}。

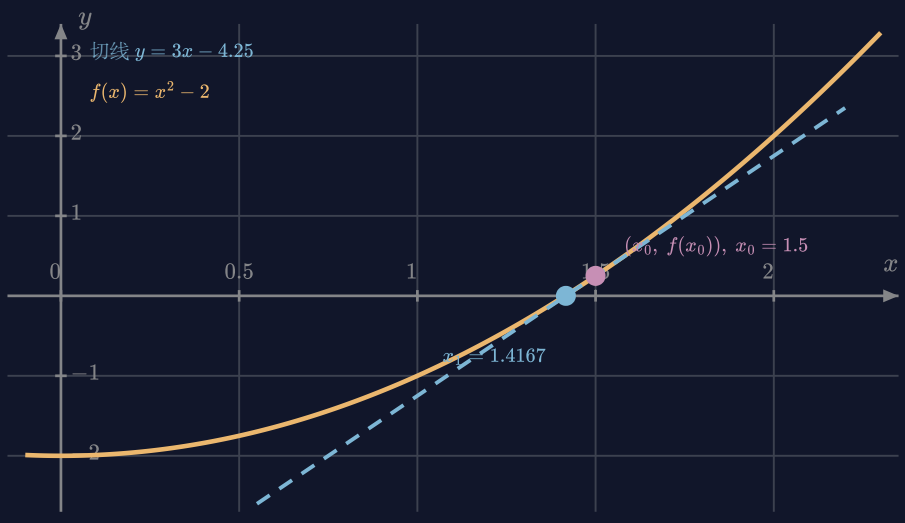


图 7 牛顿法一步：从 x_0=1.5 出发，切线与 x 轴交于 1.41667，已经逼近 $\sqrt{2}$

```python
def my_newton(f, df, x0, tol, max_iter=100):
    """牛顿-拉夫森迭代求根。"""
    x = x0
    for i in range(1, max_iter + 1):
        fx = f(x)                          # 当前函数值
        if abs(fx) < tol:
            return x, i
        dfx = df(x)                        # 当前导数值
        if abs(dfx) < 1e-12:               # 导数接近 0：切线几乎水平，方法失效
            raise ZeroDivisionError("Derivative near zero. Method failed.")
        x = x - fx / dfx
    return x, max_iter

f  = lambda x: x**2 - 2
df = lambda x: 2 * x

r_newton, iters = my_newton(f, df, x0=1.5, tol=1e-6)
print(f"Newton-Raphson Root: {r_newton:.12f} in only {iters} iterations!")
```

输出：

```text
Newton-Raphson Root: 1.414213562375 in only 4 iterations!
Bisection required 21 iterations for the same 1e-6 precision.
```

二次收敛：有效位数每步翻倍

在单根附近，牛顿法的误差满足 $e_{k+1} \approx C\, e_k^2$。把误差逐步列出来最直观：

表 4 牛顿法求 $\sqrt{2}$（从 $x_0 = 1.5$ 出发）的误差演化

迭代 k	x_k	误差 $\lvert x_k - \sqrt{2}\rvert$	正确位数
0	1.500000000000	8.6×10^{-2}	1
1	1.416666666667	2.5×10^{-3}	2–3
2	1.414215686275	2.1×10^{-6}	6
3	1.414213562375	1.6×10^{-12}	12
4	1.414213562373	$\sim 10^{-16}$	机器精度

正确位数依次是 $1 \to 3 \to 6 \to 12$ ——**每一步翻倍**，这就是二次收敛的含义。

二分法

可靠：有异号区间就一定收敛。 **慢**：线性收敛，21 步才到 10^{-6}。 不需要导数；对函数形状几乎没要求。 需要事先知道一个包住根的区间。

牛顿法

快：二次收敛，4 步就到 10^{-6}。 **脆**：强烈依赖初值与 f'。 需要解析导数（或再用差分近似，引入新误差）。 一旦 $f'(x_k) \to 0$ 就会被甩飞。

注意

牛顿法的三种典型翻车方式： ① **导数趋零**——切线几乎水平，$-f/f'$ 把你甩到天边（代码里那句 `if abs(dfx) < 1e-12` 就是在防这个）。 ② **振荡不收敛**——例如 $f(x) = \arctan x$ 从较大的 x_0 出发，迭代会在正负之间越跳越远。 ③ **收敛到"别的根"**——多根函数里，初值稍微一动，结果可能落到完全不同的根上。

实务中的解法是**混合策略**：先用二分法把根粗略夹住，再切换牛顿法冲刺。SciPy 的 `brentq` 正是这类"包围 + 超线性"的组合拳，也是实际工程里最常用的一维求根函数。

04
数值微分

数值网格与三种差分

在科学计算里，连续函数通常被放在一个**数值网格**上——定义域内一组等距离散点：

$$x_j = x_0 + j \cdot h, \qquad j = 0, 1, 2, \ldots, N$$

其中 h 是均匀步长（网格间距）。微积分里导数的定义是

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

但在离散网格上 h 不能取到无穷小，于是用邻点的函数值构造**有限差分**：

表 5　三种一阶差分格式

名称	公式	用到的点	精度阶
前向差分	$f'(x_j) \approx \dfrac{f(x_{j+1}) - f(x_j)}{h}$	$x_j,\ x_{j+1}$	$\mathcal{O}(h)$
后向差分	$f'(x_j) \approx \dfrac{f(x_j) - f(x_{j-1})}{h}$	$x_{j-1},\ x_j$	$\mathcal{O}(h)$
中心差分	$f'(x_j) \approx \dfrac{f(x_{j+1}) - f(x_{j-1})}{2h}$	$x_{j-1},\ x_{j+1}$	$\mathcal{O}(h^2)$

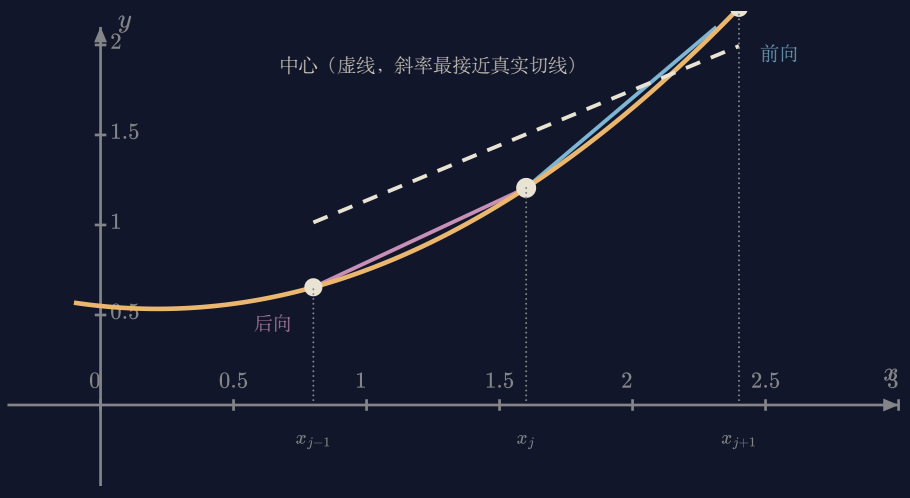


图 8　三种差分就是三条割线：中心差分用对称的两侧点，天然更准

用泰勒级数看清精度阶

为什么中心差分更好？把 f 在 x_j 处展开：

$$f(x_j+h) = f(x_j) + hf'(x_j) + \frac{h^2}{2!}f''(x_j) + \frac{h^3}{3!}f'''(x_j) + \mathcal{O}(h^4) \quad \text{(Eq. 1)}$$

$$f(x_j-h) = f(x_j) - hf'(x_j) + \frac{h^2}{2!}f''(x_j) - \frac{h^3}{3!}f'''(x_j) + \mathcal{O}(h^4) \quad \text{(Eq. 2)}$$

前向差分的精度。 把 Eq. 1 整理出 $f'(x_j)$：

$$f'(x_j) = \frac{f(x_j+h)-f(x_j)}{h} - \frac{h}{2}f''(x_j) - \cdots = \frac{f(x_{j+1})-f(x_j)}{h} + \mathcal{O}(h)$$

首项截断误差正比于 h，所以是**一阶精度 $\mathcal{O}(h)$：h 减半，误差减半。** 后向差分结构完全一样。

中心差分的精度。 用 Eq. 1 减 Eq. 2，偶数次项（含 $h^2 f''$）全部抵消：

$$f(x_j+h) - f(x_j-h) = 2hf'(x_j) + \frac{2h^3}{6}f'''(x_j) + \mathcal{O}(h^5)$$

两边除以 $2h$：

$$f'(x_j) = \frac{f(x_{j+1})-f(x_{j-1})}{2h} - \frac{h^2}{6}f'''(x_j) - \cdots = \frac{f(x_{j+1})-f(x_{j-1})}{2h} + \mathcal{O}(h^2)$$

二阶精度 $\mathcal{O}(h^2)$：h 减半，误差变为四分之一。

> **要点**
>
> 中心差分之所以免费升一阶，全靠**对称抵消**：$+h$ 和 $-h$ 的展开里，f'' 项符号相同，相减时消掉了。这个"对称性带来额外精度"的套路，稍后在积分的中点法则里会**一模一样**地再出现一次。

```python
# 对比前向、后向与中心差分
h = 0.1
x_grid = np.arange(0, 2 * np.pi, h)
y_grid = np.cos(x_grid)

# 前向差分：np.diff 求相邻元素之差
forward_diff  = np.diff(y_grid) / h
x_fwd = x_grid[:-1]

# 后向差分：数值与前向相同，但"贴"到右端点上
backward_diff = np.diff(y_grid) / h
x_bwd = x_grid[1:]

# 中心差分：跨过两格再除以 2h
central_diff  = (y_grid[2:] - y_grid[:-2]) / (2 * h)
x_cent = x_grid[1:-1]

# 精确解 d/dx cos(x) = -sin(x)
err_fwd  = np.max(np.abs(-np.sin(x_fwd)  - forward_diff))
err_bwd  = np.max(np.abs(-np.sin(x_bwd)  - backward_diff))
err_cent = np.max(np.abs(-np.sin(x_cent) - central_diff))

print(f"Max Error (Forward,  O(h)):    {err_fwd:.6f}")
print(f"Max Error (Backward, O(h)):    {err_bwd:.6f}")
print(f"Max Error (Central,  O(h^2)):  {err_cent:.6f}")
print(f"Central Difference is ~{err_fwd / err_cent:.1f}x more accurate!")
```

输出：

```text
Step size h = 0.1
Max Error (Forward Difference, O(h)):    0.049984
Max Error (Backward Difference, O(h)):   0.049970
Max Error (Central Difference, O(h^2)):  0.001666
Central Difference is ~30.0x more accurate!
```

注

代码里 `forward_diff` 和 `backward_diff` 的**数值完全相同**，都是 `np.diff(y)/h`。区别只在于你把这个斜率**归到哪个点**：归给左端点就是前向，归给右端点就是后向。所以两者的误差也几乎一样（0.049984 vs 0.049970，差别只来自端点取舍）。

把 $h = 0.1$、$\mathcal{O}(h)$ 与 $\mathcal{O}(h^2)$ 的常数代进去，30 倍这个比值并不意外：$\dfrac{h/2}{h^2/6} = \dfrac{3}{h} = 30$。理论与实测严丝合缝。

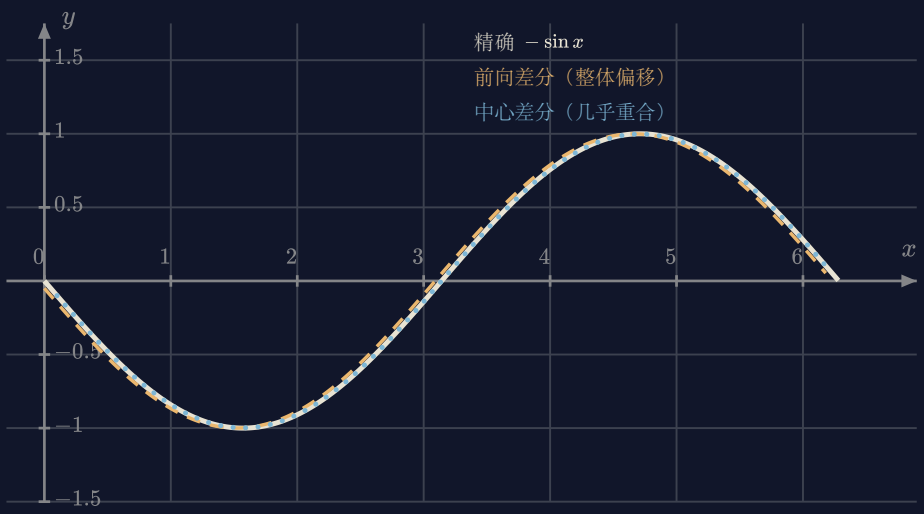


图 9 h=0.1 时的三条曲线：前向差分明显偏移，中心差分与精确解几乎分不开

误差标度：用对数图验证精度阶

要验证理论上的 $\mathcal{O}(h)$ 与 $\mathcal{O}(h^2)$，就系统地缩小 h ，看最大绝对误差怎么变。在双对数坐标下

$$\log(\text{Error}) \approx \log(C\,h^p) = p\,\log h + \log C$$

这是一条直线，**斜率就是收敛阶** p 。

```python
# 双对数误差分析：h 逐次减半
iterations = 10
h_val = 1.0
step_sizes, errors_fwd, errors_cent = [], [], []

for i in range(iterations):
    h_val /= 2.0
    step_sizes.append(h_val)

    x = np.arange(0, 2 * np.pi, h_val)
    y = np.cos(x)

    fwd = np.diff(y) / h_val                        # 前向
    errors_fwd.append(np.max(np.abs(-np.sin(x[:-1]) - fwd)))

    cent = (y[2:] - y[:-2]) / (2 * h_val)           # 中心
    errors_cent.append(np.max(np.abs(-np.sin(x[1:-1]) - cent)))

plt.loglog(step_sizes, errors_fwd,  'o-', label='Forward  (Slope ~ 1)')
plt.loglog(step_sizes, errors_cent, 'o-', label='Central  (Slope ~ 2)')
```

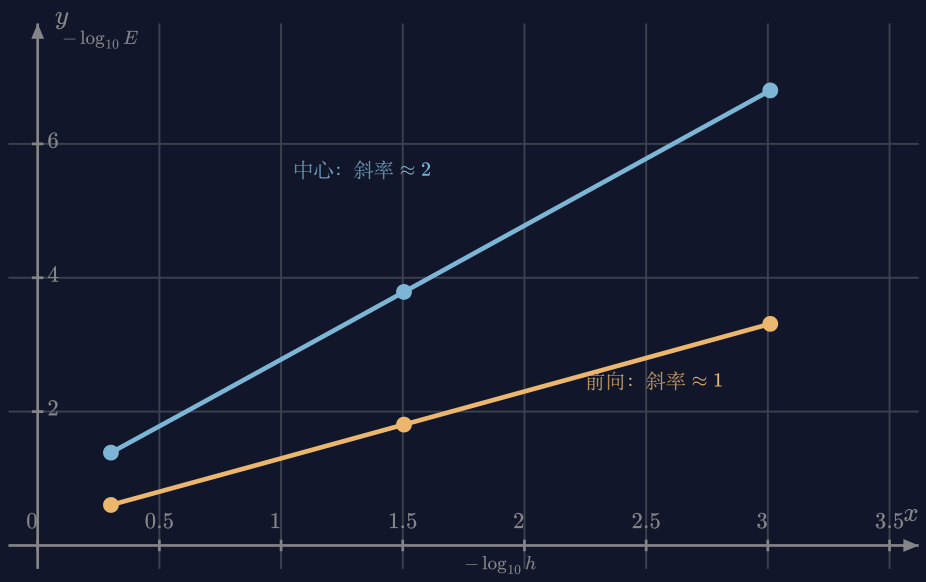


图 10 双对数收敛图（实测数据）：横轴越往右步长越细，纵轴越往上误差越小；前向斜率 1，中心斜率 2

表 6 实测最大误差（$f = \cos x$ 于 $[0, 2\pi]$）

h	前向差分误差	中心差分误差	中心/前向
$2^{-1} = 0.5$	2.48×10^{-1}	4.10×10^{-2}	6.0×
$2^{-3} = 0.125$	6.25×10^{-2}	2.60×10^{-3}	24×
$2^{-5} \approx 0.031$	1.56×10^{-2}	1.63×10^{-4}	96×
$2^{-8} \approx 0.0039$	1.95×10^{-3}	2.54×10^{-6}	768×
$2^{-10} \approx 0.00098$	4.88×10^{-4}	1.59×10^{-7}	3072×

每次 h 减半，前向误差变 $1/2$、中心误差变 $1/4$，两者的差距因此**不断拉大**。

注意

补充：h 并不是越小越好。 上图只画到 $h \approx 10^{-3}$ 就停了，这不是偶然。真实总误差由两部分组成：

$$E(h) \;\approx\; \frac{h}{2}\,|f''| \;+\; \frac{2\varepsilon}{h}$$

前一项是**截断误差**（h 越小越小），后一项是**舍入误差**（h 越小越大）。

其中 $\varepsilon \approx 2.2 \times 10^{-16}$ 是双精度机器精度。分子 $f(x+h) - f(x)$ 在 h 很小时是两个几乎相等的数相减（**灾难性抵消**），有效位数被吃光，再除以一个极小的 h，误差被放大。

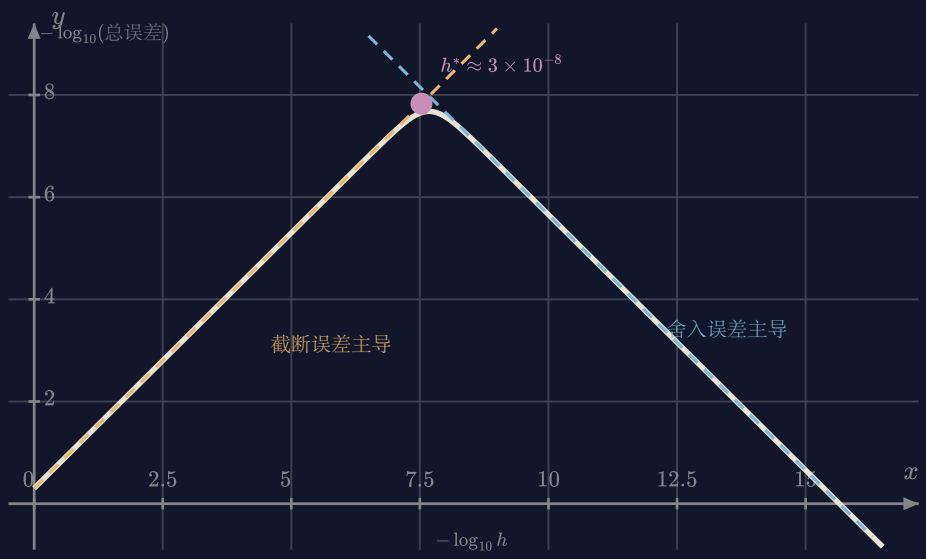


图 11 总误差的倒 U 形曲线：前向差分的最优步长约为 $\sqrt{\varepsilon} \approx 10^{-8}$，h 再小精度反而下降

提示

记住两个经验步长：**前向差分最优** $h \approx \sqrt{\varepsilon} \approx 10^{-8}$，**中心差分最优** $h \approx \varepsilon^{1/3} \approx 6 \times 10^{-6}$。想要更高精度，不要再缩 h，而要**换更高阶的格式**（如五点公式）或改用自动微分 / 复步长微分。

高阶导数

二阶导数在物理与工程里无处不在（加速度、曲率、扩散方程）。还是用泰勒级数：把 Eq. 1 与 Eq. 2 **相加**，奇数次项全部抵消：

$$f(x_j + h) + f(x_j - h) = 2f(x_j) + h^2 f''(x_j) + \frac{h^4}{12} f^{(4)}(x_j) + \mathcal{O}(h^6)$$

整理出二阶导：

$$\boxed{f''(x_j) = \frac{f(x_{j+1}) - 2f(x_j) + f(x_{j-1})}{h^2} + \mathcal{O}(h^2)}$$

这就是标准的**二阶中心差分二阶导公式**，用三个点 (x_{j-1}, x_j, x_{j+1})，截断误差 $\mathcal{O}(h^2)$。系数 $(1, -2, 1)$ 值得背下来——它是所有差分法解扩散方程、泊松方程的基石。

```python
h = 0.05
x = np.arange(0, 2 * np.pi, h)
y = np.cos(x)

# 二阶导近似：(y[j+1] - 2*y[j] + y[j-1]) / h^2
second_diff = (y[2:] - 2 * y[1:-1] + y[:-2]) / (h**2)
x_interior  = x[1:-1]                    # 注意：两端各丢一个点

exact_second = -np.cos(x_interior)       # d²/dx² cos(x) = -cos(x)
max_second_err = np.max(np.abs(exact_second - second_diff))
print(f"Max Error in Second Derivative (O(h^2)): {max_second_err:.6e}")
```

输出：

```text
Step size h = 0.05
Max Error in Second Derivative (O(h^2)): 2.083086e-04
```

验算一下理论值：截断误差首项是 $\frac{h^2}{12}|f^{(4)}| = \frac{0.05^2}{12} \times 1 \approx 2.08 \times 10^{-4}$，和实测 2.083086×10^{-4} **完全吻合**。这种"理论预测 ↔ 数值实测"的对拍，是检验代码写对没写对的最好办法。

> **注**
>
> 注意 `second_diff` 的长度比 `x` 少 2，因为**端点没有左右邻居**。这是有限差分的通病：边界点需要单独处理（用单侧格式，或者加 ghost point）。写向量化代码时，切片长度对不上是最常见的 bug 来源。

微分对噪声极其敏感

传感器、加速度计、光学编码器拿到的真实数据总是带噪声。考虑一个被高频噪声污染的干净信号：

$$f_{\epsilon,\omega}(x) = \cos x + \epsilon \sin(\omega x)$$

其中 $\epsilon = 0.01$ 是很小的噪声幅度（仅为信号的 1%），$\omega = 100$ 是很高的角频率。解析求导：

$$f'_{\epsilon,\omega}(x) = -\sin x + \epsilon\, \omega \cos(\omega x)$$

看导数里噪声项的幅度：

$$\epsilon \cdot \omega = 0.01 \times 100 = 1.0$$

原始数据里肉眼不可见的 1% 噪声，在导数里变成了幅度 1.0——和真实信号一样大。

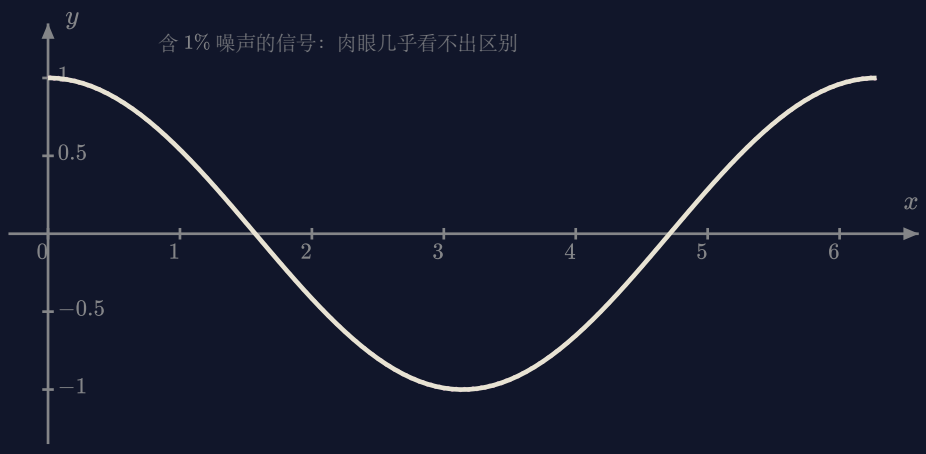


图 12 原始信号：cos(x) 与 cos(x)+0.01·sin(100x) 几乎重合

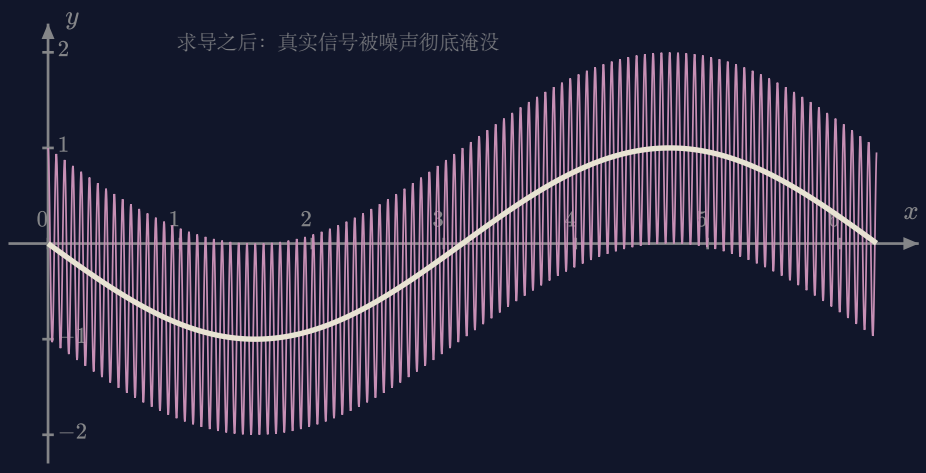


图 13 导数：噪声幅度 ε·ω = 1.0，与真实导数 -sin(x) 同量级

在有限差分里情况更糟。噪声进入分子后要除以 h ：

$$\frac{\Delta y_{\text{noise}}}{h}$$

h **越小，噪声被放大得越厉害。** 这与前面"截断误差要求 h 小"的诉求直接冲突。

要点

微分是放大器，积分是平滑器。 微分把高频噪声按 ω 倍放大；积分则把随机噪声互相抵消掉。所以实验数据处理的铁律是： ① 先**滤波/平滑**（Savitzky–Golay、低通滤波、样条拟合），再求导； ② 或者干脆**不要数值求导**——先对数据拟合一个解析模型，再对模型解析求导。

05
数值积分

数值积分要逼近定积分

$$I = \int_a^b f(x)\,dx$$

几何上，它就是曲线与 x 轴之间的**带符号面积**。所有方法的套路都一样：**把 $[a,b]$ 切成小块，在每块上用一个容易算面积的简单形状替代曲线，然后把面积加起来。** 区别只在于"简单形状"是什么。

把区间 $[a,b]$ 分成 $n-1$ 个等宽子区间（即 n 个网格点）：

$$h = \frac{b-a}{n-1}$$

左/右黎曼和：用矩形

最朴素的做法，用每个子区间**某一端**的函数值作为矩形高度：

$$\text{左黎曼和：}\ \int_a^b f\,dx \approx \sum_{i=0}^{n-2} h\,f(x_i), \qquad \text{右黎曼和：}\ \int_a^b f\,dx \approx \sum_{i=1}^{n-1} h\,f(x_i)$$

单个子区间的误差是 $\mathcal{O}(h^2)$；但一共有 $N=(b-a)/h$ 个子区间，累加之后**全局误差是 $\mathcal{O}(h)$**——这个"单区间降一阶"的换算关系，对后面每种方法都成立。

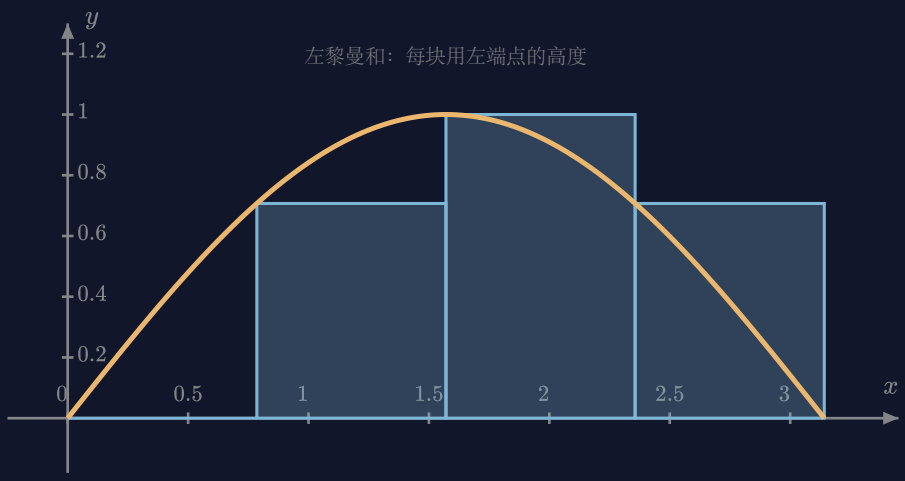


图 14 左黎曼和逼近 ∫₀^π sin x dx：矩形顶端与曲线之间的空隙就是误差

中点法则：对称性再次救场

中点法则改用**子区间中点**的函数值作为矩形高度：

$$y_i = \frac{x_i + x_{i+1}}{2}, \qquad \int_a^b f\,dx \approx \sum_{i=0}^{n-2} h\, f(y_i)$$

它是**二阶精度**的，因为对称性让泰勒展开里的低阶误差项互相抵消——和中心差分完全同一个机制：矩形在中点左边少算的那块，恰好被右边多算的那块补回来。

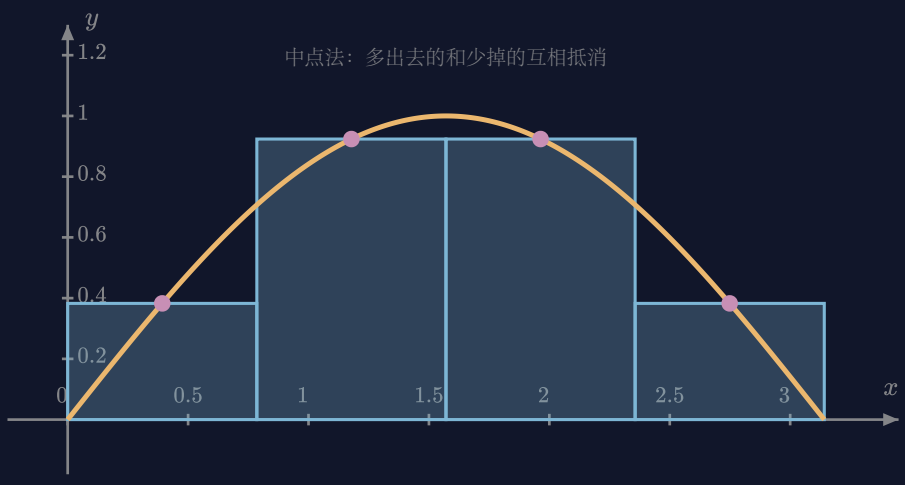


图 15 中点法则：矩形顶端穿过曲线，左右两侧误差相消

> **注意**
>
> 中点法则有个现实限制：**它需要中点处的函数值**。如果你拿到的是实验测得的一张表格（只有网格点上的数据），中点的值根本不存在，这个方法就用不了。这正是梯形法则在实验数据处理中更常用的原因。

梯形法则：用直线

梯形法则在每个子区间上把函数近似为**连接两端点的直线**（也就是线性插值），算梯形面积：

$$\text{Area}_i = \frac{h}{2}[f(x_i) + f(x_{i+1})]$$

全部加起来：

$$\int_a^b f\,dx \approx \sum_{i=0}^{n-2} \frac{h}{2}[f(x_i) + f(x_{i+1})]$$

注意到每个**内点** $x_1, \ldots, x_{n-2}$ 都被相邻两个梯形共用、**数了两次**，而两个端点各只数一次，于是可以优化成只扫一遍数组的形式：

$$\boxed{\int_a^b f\,dx \approx \frac{h}{2}\left[f(x_0) + 2\sum_{i=1}^{n-2} f(x_i) + f(x_{n-1})\right]}$$

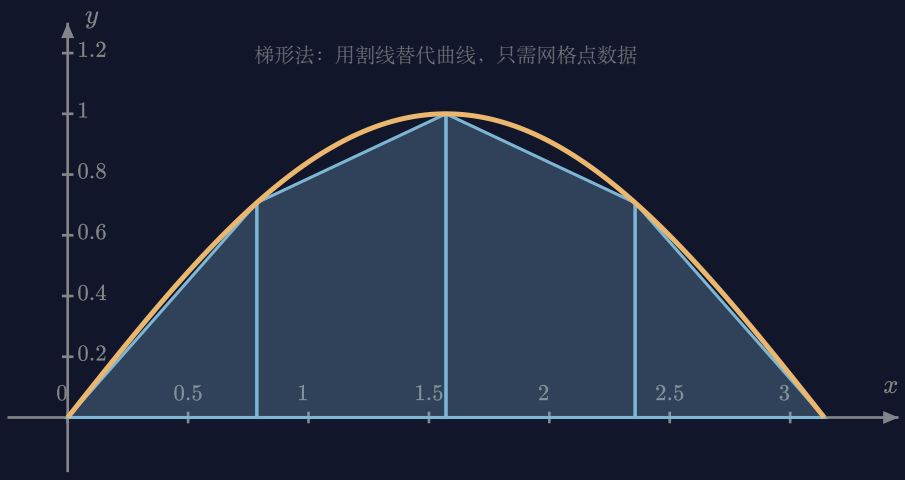


图 16 梯形法则：斜边贴着曲线，凹函数处会系统性低估面积

梯形法则同样是**二阶精度**，但只用给定的网格点，因此可以**直接用于实验表格数据**。缺点是曲率大的地方，用直线替代曲线不够好。

辛普森 1/3 法则：用抛物线

与其用直线段，不如用**抛物线**：取三个连续点 (x_{i-1}, x_i, x_{i+1})（跨度 $2h$）拟合一条二次曲线，再把抛物线在该区间上**精确积分**。对所有相邻点对求和：

$$\boxed{\int_a^b f\,dx \approx \frac{h}{3}\left[f(x_0) + 4\sum_{i\ \text{odd}} f(x_i) + 2\sum_{i\ \text{even}} f(x_i) + f(x_{n-1})\right]}$$

系数模式 $1, 4, 2, 4, 2, \ldots, 4, 1$ 值得记牢。使用条件：**网格点数 n 必须是奇数**（这样子区间数 $n-1$ 才是偶数，能两两配对）。

辛普森法则是**四阶精度** $\mathcal{O}(h^4)$，因为对称性让三次项也抵消了——所以它对**所有次数 ≤ 3 的多项式都是精确的**，尽管它只用了抛物线去拟合。这是个很反直觉但极其有用的事实。

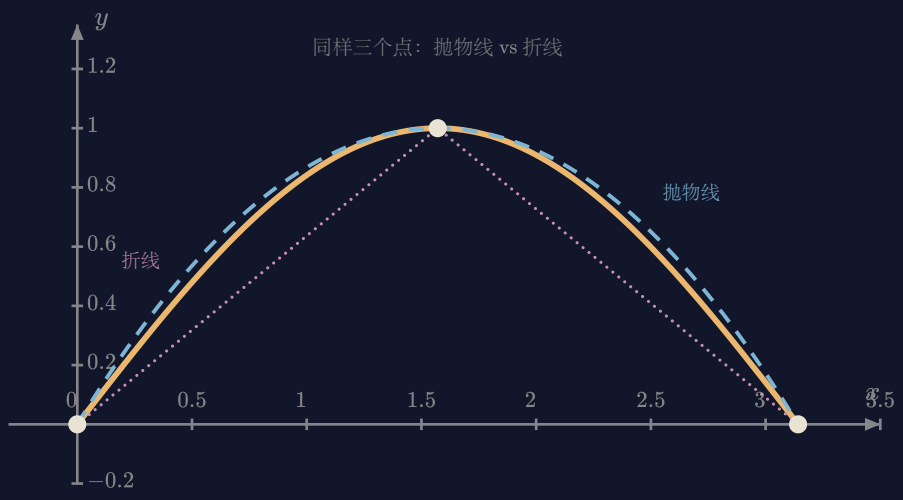


图 17 辛普森法则的几何：抛物线（虚线）贴合 sin x 远好于梯形折线（点线）

四种方法同场竞技

用同一个算例：$\int_0^{\pi} \sin x \, dx = 2$，取 $n = 11$ 个网格点（$h = \pi/10 \approx 0.3142$）。

```python
a, b, n = 0, np.pi, 11
h = (b - a) / (n - 1)
x = np.linspace(a, b, n)
f = np.sin(x)

I_riemannL = h * np.sum(f[:-1])                          # 左黎曼和
I_riemannR = h * np.sum(f[1:])                           # 右黎曼和
x_mid = (x[:-1] + x[1:]) / 2.0
I_mid  = h * np.sum(np.sin(x_mid))                       # 中点法则
I_trap = (h / 2.0) * (f[0] + 2 * np.sum(f[1:-1]) + f[-1])     # 梯形法则
I_simp = (h / 3.0) * (f[0] + 4 * np.sum(f[1:-1:2])
                           + 2 * np.sum(f[2:-1:2]) + f[-1])   # 辛普森法则
```

输出汇总：

表 7 $\int_0^{\pi} \sin x\,dx$（精确值 $=2$）在 $n=11$ 下的对比

方法	积分值	误差	精度阶
左黎曼和	1.98352354	$+1.65\times10^{-2}$	$\mathcal{O}(h)$
右黎曼和	1.98352354	$+1.65\times10^{-2}$	$\mathcal{O}(h)$
中点法则	2.00824841	-8.25×10^{-3}	$\mathcal{O}(h^2)$
梯形法则	1.98352354	$+1.65\times10^{-2}$	$\mathcal{O}(h^2)$
辛普森法则	2.00010952	$\mathbf{-1.10\times10^{-4}}$	$\mathcal{O}(h^4)$

要点

这张表里有两个"巧合"必须解释清楚，否则容易学歪。

① **为什么左、右黎曼和完全相等？** 因为 $\sin 0=\sin\pi=0$，两个端点的贡献都是 0，而中间项两边一样，所以数值相同。换个函数（比如 $\int_0^2 x^2dx$）立刻就不相等了——练习第 4 题会亲眼看到。

② **为什么梯形法也等于 1.98352354？** 同理：梯形公式比左黎曼和多出的就是 $\frac{h}{2}[f(b)-f(a)]$，这里恰好是 0。所以别被这个巧合骗了，**梯形法确实是二阶、黎曼和确实是一阶**。要验证这一点，就把 h 减半再算一次：黎曼和的误差减半，梯形法的误差变成四分之一。

注

注意中点法与梯形法的误差**符号相反**（-8.25×10^{-3} 对 $+1.65\times10^{-2}$），而且大小正好是 $1:2$。这不是巧合：对凹函数，梯形的割线在曲线**下方**（低估），中点的矩形顶端在曲线**上方**（高估），且理论上 $E_{\text{trap}}=-2E_{\text{mid}}$。把两者按 $\frac{2}{3}$ 中点 $+\frac{1}{3}$ 梯形加权平均，误差正好抵消——**这就是辛普森法则的另一种推导方式**。

SciPy 的积分工具箱

`scipy.integrate` 对**离散数组**和**解析函数**分别给了优化实现：

trapezoid

对给定的离散数组 y 与 x 计算梯形积分总值。处理实验数据的首选。

cumulative_trapezoid

计算**累积**积分曲线

$$F(X) = \int_{x_0}^{X} f(t)\,dt$$

参数与 `trapezoid` 相同，加 `initial=0` 可让输出与输入等长。

quad

面向**可调用函数**的高精度自适应方法，返回积分值**和误差上界**两个数。

```python
from scipy.integrate import trapezoid, cumulative_trapezoid, quad

I_scipy_trap = trapezoid(f, x)
print(f"scipy.integrate.trapezoid: {I_scipy_trap:.8f}")

# 累积积分：∫₀ˣ sin t dt 的精确解是 1 - cos(x)
x_fine = np.arange(0, np.pi, 0.01)
F_approx = cumulative_trapezoid(np.sin(x_fine), x_fine, initial=0)

# 自适应高精度积分
I_quad, est_err = quad(np.sin, 0, np.pi)
print(f"quad result: {I_quad:.14f}   estimated error bound: {est_err:.2e}")
```

输出：

```text
scipy.integrate.trapezoid: 1.98352354 (Matches manual: 1.98352354)

scipy.integrate.quad result: 2.00000000000000
Exact Analytical Error:      0.00e+00
SciPy Estimated Error Bound: 2.22e-14
```

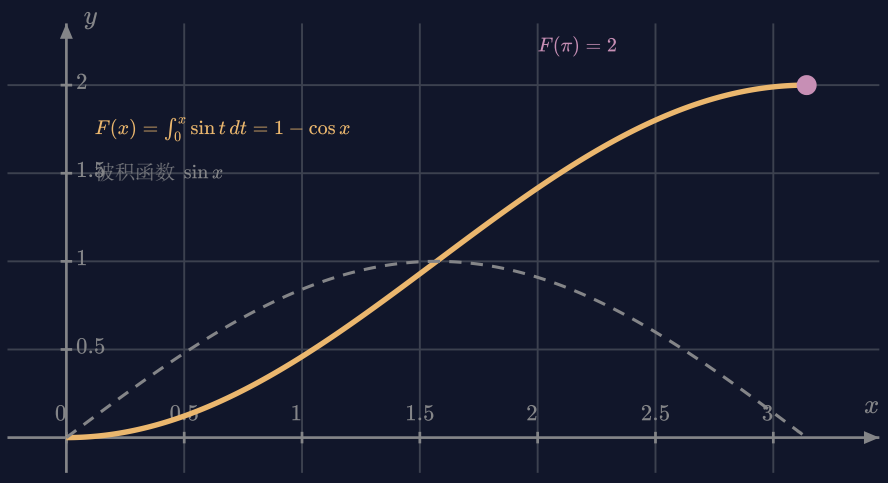


图 18 累积积分：cumulative_trapezoid 的输出与解析解 1-cos x 肉眼不可分辨

要点

为什么 `quad` 能准到 10^{-14}，而手写辛普森只有 10^{-4}？ 两个原因：① `quad` 用的是**高斯–克朗罗德求积**，节点位置经过最优化，m 个节点能精确积分 $2m-1$ 次多项式，而不是固定等距网格；② 它是**自适应**的——会自动在函数变化剧烈的地方加密采样，并给出误差估计 `est_err`。**什么时候用哪个：手上是一个能求值的函数 → `quad`；手上是一张离散数据表 → `trapezoid` / `simpson`。**

表 8 积分方法选型速查

情形	推荐方法	理由
有解析函数、要高精度	`quad`	自适应 + 误差上界
离散数据、点数为奇数	辛普森 / `simpson`	四阶精度，几乎免费
离散数据、点数为偶数或带噪声	梯形 / `trapezoid`	稳健，不放大噪声
需要积分曲线而非单值	`cumulative_trapezoid`	直接给出 $F(x)$
只有中点可测	中点法则	二阶精度

06

练习与完整解答

注

原讲义要求这些题目**手写公式实现**，内置函数只用来验证结果。下面给出每题的推导过程、参考答案，以及"这道题想考你什么"。建议先自己做一遍再看答案。

第 1 题：插值

已知数据，用线性插值、三次样条插值、牛顿多项式插值分别估计 $f(3)$ 并比较：

x	2	4	6
$f(x)$	5	9	15

线性插值。 $x=3$ 落在 $[2,4]$：

$$\hat{f}(3) = 5 + \frac{9-5}{4-2}(3-2) = 5 + 2 = \mathbf{7}$$

牛顿多项式。 先算差商表：$f[2,4] = \frac{9-5}{2} = 2$，$f[4,6] = \frac{15-9}{2} = 3$，$f[2,4,6] = \frac{3-2}{6-2} = 0.25$。于是

$$P(x) = 5 + 2(x-2) + 0.25(x-2)(x-4)$$

$$P(3) = 5 + 2 + 0.25 \times (1) \times (-1) = \mathbf{6.75}$$

（三点唯一确定一条抛物线，所以拉格朗日插值会给出**完全相同**的 6.75。）

自然三次样条。 $h=2$，$M_0 = M_2 = 0$，内点方程

$$\frac{2h}{3} M_1 = \frac{15-9}{2} - \frac{9-5}{2} = 1 \implies M_1 = 0.75$$

代入分段公式，在 $x=3$（即 $[2,4]$ 段的中点）：

$S(3) = \frac{5+9}{2} + \frac{1}{6}\left(\frac{1^3}{2} - 2\right) \times 0.75$

$S(3) = 7 - 0.1875 = \mathbf{6.8125}$

（第一项 $\frac{5+9}{2}=7$ 是线性部分，第二项是二阶导带来的修正。）

表 9 第 1 题三种方法的对比

方法	$\hat{f}(3)$	曲线形状	说明
线性插值	7	折线	只用左右两点，忽略数据的上凸趋势
自然三次样条	6.8125	分段三次，C^2	受自然边界条件"压平"，略低于抛物线
牛顿 / 拉格朗日	6.75	单条抛物线	完整利用三点的曲率信息

要点

三个答案都在 $6.75\sim7$ 之间，差别不到 4%。**这题想考的是"方法不同，答案就不同"**——没有哪个是"正确答案"，取决于你对底层函数光滑性的假设。数据明显上凸（增量 $4\to6$），线性插值忽略了这个趋势所以估得最高。

第 2 题：求根

用二分法与牛顿法求 $f(x)=x^2-5$ 在 $[2,3]$ 上的根（精确值 $\sqrt{5}=2.2360679775$）。

二分法。 $f(2)=-1<0$，$f(3)=4>0$，异号，可以起步。

表 10 二分法在不同容差下的表现

容差 tol	近似根	迭代次数
10^{-2}	2.234375	6
10^{-4}	2.236083984	12
10^{-6}	2.236067772	20

验算迭代次数：$k\approx\log_2\frac{b-a}{\text{tol}}=\log_2\frac{1}{10^{-6}}\approx19.9$，取 20 ✓

牛顿法。 $f'(x)=2x$，迭代式

$$x_{k+1}=x_k-\frac{x_k^2-5}{2x_k}=\frac{1}{2}\left(x_k+\frac{5}{x_k}\right)$$

（这正是古巴比伦人开平方的算法！）从 $x_0=2.5$ 出发：

表 11 牛顿法迭代过程（$x_0 = 2.5$）

k	x_k	误差 $\lvert x_k - \sqrt{5}\rvert$
0	2.500000000000	2.6×10^{-1}
1	2.250000000000	1.4×10^{-2}
2	2.236111111111	4.3×10^{-5}
3	2.236067977916	4.2×10^{-10}
4	2.236067977500	0（机器精度）

3 步就超过了二分法 20 步的精度。误差序列 $10^{-1} \to 10^{-2} \to 10^{-5} \to 10^{-10}$，指数近似翻倍——教科书式的二次收敛。

第 3 题：数值微分

对 $f(x) = x^2 + 3x$，用 $h = 0.1$ 的中心差分与二阶导公式估计 $f'(2)$ 和 $f''(2)$。

先准备三个函数值：$f(1.9) = 3.61 + 5.7 = 9.31$，$f(2) = 4 + 6 = 10$，$f(2.1) = 4.41 + 6.3 = 10.71$。

$$f'(2) \approx \frac{f(2.1) - f(1.9)}{2h} = \frac{10.71 - 9.31}{0.2} = \frac{1.4}{0.2} = \mathbf{7}$$

$$f''(2) \approx \frac{f(2.1) - 2f(2) + f(1.9)}{h^2} = \frac{10.71 - 20 + 9.31}{0.01} = \frac{0.02}{0.01} = \mathbf{2}$$

解析解：$f'(x) = 2x + 3 \Rightarrow f'(2) = 7$；$f''(x) = 2$。**两个结果都是精确的，误差为零。**

要点

误差为零不是运气。 中心差分的截断误差首项是 $-\frac{h^2}{6} f'''(x)$，而 f 是二次多项式，$f''' \equiv 0$，所以误差项整体消失。同理二阶导公式的误差首项含 $f^{(4)} \equiv 0$。

一般规律：中心差分对次数 ≤ 2 的多项式精确；二阶导中心公式对次数 ≤ 3 的多项式精确。这也是调试差分代码的好办法——**先拿一个低次多项式测试，结果应该精确到机器精度**；如果不是，说明代码写错了。

第 4 题：数值积分

用右黎曼和、中点法则、梯形法则、辛普森法则，取 $n = 21$ 逼近 $\displaystyle\int_0^2 (x^2 + 1)\,dx$。

精确值：$\left[\frac{x^3}{3} + x\right]_0^2 = \frac{8}{3} + 2 = \frac{14}{3} = 4.6\overline{6}$，步长 $h = \frac{2-0}{21-1} = 0.1$。

表 12 第 4 题四种方法的结果（精确值 $14/3 \approx 4.66666667$）

方法	积分值	误差	相对误差
左黎曼和（参考）	4.47000000	$+1.97 \times 10^{-1}$	4.2%
右黎曼和	4.87000000	-2.03×10^{-1}	4.4%
中点法则	4.66500000	$+1.67 \times 10^{-3}$	0.036%
梯形法则	4.67000000	-3.33×10^{-3}	0.071%
辛普森法则	4.66666667	$\sim 10^{-16}$	机器精度

要点

这道题是上一章那两个"巧合"的解药。

① **左 ≠ 右了。** $f(0)=1 \neq f(2)=5$，所以左右黎曼和相差 $h[f(b)-f(a)] = 0.1 \times 4 = 0.4$，正好是 $4.87 - 4.47$✓ 而且两者**误差符号相反**，因为 f 单调递增：左端点系统性低估，右端点系统性高估。

② **辛普森精确到机器精度。** 因为 $f(x)=x^2+1$ 是二次多项式，而辛普森法则对次数 ≤ 3 的多项式**精确成立**。这正好印证了"四阶精度"的含义：误差项含 $f^{(4)} \equiv 0$。

③ **中点与梯形误差比仍是 $1:2$、符号相反**（$+1.67 \times 10^{-3}$ vs -3.33×10^{-3}），和理论 $E_{\text{trap}} = -2E_{\text{mid}}$ 完全一致。

提示

自检清单：做完题一定要验这三件事—— ① **量纲/量级对不对**（积分值应该在 $\min f \cdot (b-a)$ 与 $\max f \cdot (b-a)$ 之间）； ② **低次多项式应当精确**（辛普森对三次以内、中心差分对二次以内）； ③ **把 h 减半重算一次**，误差应按 2^p 下降——这是验证精度阶最硬的证据。

07
速查卡

核心公式一览

表 13 本讲全部公式速查

主题	公式	精度
线性插值	$\hat{f}(x)=y_i+\dfrac{y_{i+1}-y_i}{x_{i+1}-x_i}(x-x_i)$	C^0
拉格朗日	$L(x)=\sum y_iP_i(x),\ P_i=\prod_{j\neq i}\dfrac{x-x_j}{x_i-x_j}$	全局 $n-1$ 次
牛顿差商	$a_k=f[x_0,\dots,x_k]$	全局 $n-1$ 次
二分法	$m=\dfrac{a+b}{2}$，按符号砍半	线性
牛顿法	$x_{k+1}=x_k-\dfrac{f(x_k)}{f'(x_k)}$	二次
前向差分	$\dfrac{f_{j+1}-f_j}{h}$	$\mathcal{O}(h)$
中心差分	$\dfrac{f_{j+1}-f_{j-1}}{2h}$	$\mathcal{O}(h^2)$
二阶导	$\dfrac{f_{j+1}-2f_j+f_{j-1}}{h^2}$	$\mathcal{O}(h^2)$
黎曼和	$h\sum f(x_i)$	$\mathcal{O}(h)$
中点法	$h\sum f\left(\frac{x_i+x_{i+1}}{2}\right)$	$\mathcal{O}(h^2)$
梯形法	$\dfrac{h}{2}\left[f_0+2\sum_{i=1}^{n-2}f_i+f_{n-1}\right]$	$\mathcal{O}(h^2)$
辛普森	$\dfrac{h}{3}[f_0+4\sum_{\text{odd}}f_i+2\sum_{\text{even}}f_i+f_{n-1}]$	$\mathcal{O}(h^4)$

一眼看懂"阶"的含义

表 14 h 减半时误差如何变化

精度阶	$h \to h/2$ 误差变为	想把误差降到 1/100 需要	代表方法
$\mathcal{O}(h)$	$1/2$	h 缩小 100 倍	前向/后向差分、黎曼和
$\mathcal{O}(h^2)$	$1/4$	h 缩小 10 倍	中心差分、中点法、梯形法
$\mathcal{O}(h^4)$	$1/16$	h 缩小 3.2 倍	辛普森法则

常见坑清单

用残差判收敛 $|f(x)|$ 小不等于找到根。务必同时检查 `ier`、`mesg` 与迭代次数（回看 $1/x$ 那个 10^{83}）。

高次多项式插值 等距节点 + 高次 = 龙格振荡。点多就用分段三次样条。

差分步长一味取小 $h < 10^{-8}$ 后舍入误差主导，总误差反而上升。

对噪声数据直接求导 噪声被放大 ω 倍。先平滑或先拟合模型再求导。

忘了辛普森要奇数点 n 为偶数时子区间无法两两配对，公式失效。

切片长度对不上 中心差分丢 2 个点、前向差分丢 1 个点，画图时 x 与 y 长度必须匹配。

浮点数用 == 比较 求根结果是 -1.0000000000000022 而不是 -1，永远用 `np.isclose`。

外插 插值函数在数据范围之外不可信，高次多项式尤其危险。

方法选型决策树

01 我要"点之间"的值

数据干净且点少 → 牛顿/拉格朗日；点多或要光滑导数 → 三次样条；只要个粗略值 → 线性插值。

02 我要解 $f(x) = 0$

能写出 f' 且初值不错 → 牛顿法；只知道一个异号区间 → 二分法；都想要 → `brentq`（包围 + 超线性）。

03 我要斜率

优先中心差分；端点用单侧格式；数据带噪声 → 先平滑或拟合再求导。

04 我要面积

有函数 → `quad`；有离散表且点数为奇数 → 辛普森；否则 → 梯形；要整条积分曲线 → `cumulative_trapezoid`。

要点

如果这一讲只带走一句话： 所有数值方法都是"**用简单形状替代真实对象**"，而**精度阶**就是这笔交易的定价表。知道自己在用什么替代、代价是几阶、什么时候这笔交易会崩——比记住任何一个公式都重要。